This project implements a simplified File System. It provides basic functionality to create, write, read and list files and directories inside a single disk.
The helper functions are inside utilities.c, they are used in fs.c.
Here we can see that all of these functions work as expected:

- mkfs
- mkdir_fs
- create_fs
- write_fs
- read_fs
- delete_fs
- rmdir_fs
- ls_fs

Inode is reused when the directory is deleted, and new one was created (inode2).

```
root@vm_docker:/home/ubuntu/hostvolume/HW3# make clean
rm -f minifs disk.img tests/output.txt
root@vm_docker:/home/ubuntu/hostvolume/HW3# make
gcc -Wall -Wextra -g fs.c utilities.c main.c -o minifs
root@vm_docker:/home/ubuntu/hostvolume/HW3# make run
chmod +x tests/commands.txt
tee tests/output.txt < tests/commands.txt | while read -r cmd; do \
        echo "\$ ${cmd}"; \
        eval "${cmd}"; \
done | tee -a tests/run_log.txt
$ ./minifs mkfs disk.img
Created filesystem in disk.img
$ ./minifs mkdir_fs /docs
Directory created.
$
$ ./minifs mkdir_fs /to_be_deleted
Directory created.
$ ./minifs ls_fs /to_be_deleted
$ ./minifs ls_fs /
 - docs (inode 1)
 - to_be_deleted (inode 2)
$ ./minifs rmdir_fs /to_be_deleted
Directory deleted.
$ ./minifs ls_fs /
 - docs (inode 1)
$
$ ./minifs mkdir_fs /docs/folder1
Directory created.
$ ./minifs ls_fs /
 - docs (inode 1)
$
$ ./minifs create_fs /docs/b.txt
File created.
$ ./minifs ls_fs /docs
 - folder1 (inode 2)
 - b.txt (inode 3)
$
$ ./minifs create_fs /docs/folder1/a.txt
File created.
$ ./minifs ls_fs /docs/folder1
 - a.txt (inode 4)
$
$ ./minifs write_fs /docs/b.txt "Hello b.txt"
Write successful.
$ ./minifs write_fs /docs/folder1/a.txt "Hello a.txt"
Write successful.
$ ./minifs ls_fs /
 - docs (inode 1)
$ ./minifs ls_fs /docs
 - folder1 (inode 2)
 - b.txt (inode 3)
$
$ ./minifs read_fs /docs/b.txt
Read (11 bytes): Hello b.txt
$ ./minifs read_fs /docs/folder1/a.txt
Read (11 bytes): Hello a.txt
$
$ ./minifs delete_fs /docs/folder1/a.txt
File deleted.
$ ./minifs ls_fs /docs/folder1
$ ./minifs ls_fs /docs
 - folder1 (inode 2)
 - b.txt (inode 3)
$ ./minifs rmdir_fs /docs/folder1
Directory deleted.
$ ./minifs ls_fs /docs
 - b.txt (inode 3)
$
root@vm_docker:/home/ubuntu/hostvolume/HW3#
```

Figure 1. Terminal output (also in run_log.txt)

When the syntax is wrong, or we try to access to non-existent path we get error. Since to_be-deleted is inside root folder, it should be deleted from there. But if we try to access /docs/to_be_deleted, it will print error. similar thing happens for all other commands.

```
root@vm_docker:/home/ubuntu/hostvolume/HW3# make clean
rm -f minifs disk.img tests/output.txt
root@vm_docker:/home/ubuntu/hostvolume/HW3# make
gcc -Wall -Wextra -g fs.c utilities.c main.c -o minifs
root@vm_docker:/home/ubuntu/hostvolume/HW3# make run
chmod +x tests/commands.txt
tee tests/output.txt < tests/commands.txt | while read -r cmd; do \
        echo "\$ ${cmd}"; \
        eval "${cmd}"; \
done | tee -a tests/run_log.txt
$ ./minifs mkfs disk.img
Created filesystem in disk.img
$ ./minifs mkdir_fs /docs
Directory created.
$
$ ./minifs mkdir_fs /to_be_deleted
Directory created.
$ ./minifs ls_fs /to_be_deleted
$ ./minifs ls_fs /
 - docs (inode 1)
 - to_be_deleted (inode 2)
$ ./minifs rmdir_fs /docs/to_be_deleted
Directory delete failed.
$ ./minifs ls_fs /
 - docs (inode 1)
 - to_be_deleted (inode 2)
$
```

Figure 2. Erroneous rm_dir

Questions

How are directory entries stored/searched?

Directory entries are stored using given DirectoryEntry struct. When new entry is added *int add_to_dir(int dir_ino, const char *name, int child_ino)* function is used. The function tries to find space in one of the first four blocks of the directory to add a new name/inode pair. If needed, it allocates a new block. If a free spot is found, it writes the new directory entry and saves it to disk. Otherwise, it reports failure.

*int find_in_dir(int dir_ino, const char *name)* : this function searches the contents of a directory (stored in up to 4 direct blocks) for a file or directory with a given name. If it finds it, it returns the inode number. If not, it returns -1.

What happens when the disk or inode table is full?

If the disk is full (no free data blocks in bitmap), then attempt to allocate a block using alloc_block() returns -1, and the command (create_fs, write_fs) will fail.

Similarly, if the inode table is full (all is_valid entries are 1), then alloc_inode() also returns -1, preventing new files or directories from being created. These checks ensure the system avoids memory corruption or writing past the bounds of allocated structures.

How is double allocation prevented?

Double allocation is prevented using a block bitmap, where each bit represents the allocation status of a block. When a block is allocated via alloc_block(), its corresponding bit in the bitmap is set to 1. When a block is freed, its bit is cleared to 0. This ensures that no two files or directories point to the same block. Similarly, inode allocation sets is_valid = 1, and only inodes with is_valid = 0 are available for reuse.

Describe one encountered error and your solution.

Initially, I did create_fs for only root. So all created files are under the root directory. That is why I was getting a segmentation fault when trying command /docs/folder1/a.txt. To solve that, I adde traverse_path() function to find the parent's inode, and then use that instead of the default root.